

FRAUD AWARE E-WALLET SYSTEM

A PROJECT REPORT

Submitted by

DARSH TYAGI [Reg No: RA2411028030025]

RISHAV KUAMR GUPTA [Reg No: RA2411028030032]

SUSMITA SAHOO [Reg No: RA2411028030041]

Under the guidance of

Dr. ANIRUDHA GAIKWAD

(Professor, Department of Computer Science & Engineering)

in partial fulfillment for the award of the degree

of

BACHELOR OF TECHNOLOGY

in

COMPUTER SCIENCE & ENGINEERING

of

FACULTY OF ENGINEERING AND TECHNOLOGY



SRM INSTITUTE OF SCIENCE & TECHNOLOGY, NCR CAMPUS

MAY 2026

SRM INSTITUTE OF SCIENCE & TECHNOLOGY

(Under Section 3 of UGC Act, 1956)

BONAFIDE CERTIFICATE

Certified that this project report titled “FRAUD AWARE E-WALLET SYSTEM” is the bonafide work of “ DARSH TYAGI [Reg No: RA2411028030025], RISHAV KUMAR GUPTA [Reg No: RA2411028030032], SUSMITA [RegNo:RA2411028030041] , who carried out the project work under my supervision. Certified further, that to the best of my knowledge the work reported herein does not form any other project re- port or dissertation on the basis of which a degree or award was conferred on an earlier occasion on this or any other candidate.

SIGNATURE

Dr. ANIRUDHA GAIKWAD
GUIDE
Professor Dept. of Computer Sciene
& Engi- neering

SIGNATURE

Dr. R.P.Mahapatra
HEAD OF THE DEPARTMENT
Dept. of Computer Science & Engi-
neering

Signature of the Internal Examiner

Signature of the External Examiner

ACKNOWLEDGMENT

I would like to express my deepest gratitude to my guide, Dr. ANIRUDHA GAIKWAD for his valuable guidance, consistent encouragement, personal caring, timely help and providing me with an excellent atmosphere for doing research. All through the work, in spite of his busy schedule, he has extended cheerful and cordial support to me for completing this research work.

Author

TABLE OFCONTENT

S.NO.	Title	Page No.
1	Abstract	
2	Introduction	
3	Literature Review	
4	System Analysis	
5	System Design	
6	Implementation	
7	Screenshots / Output	
8	Testing	
9	Results and Discussion	
10	Conclusion & Future Scope	
11	References	
12	Appendix	

ABSTRACT

With the rapid growth of digital transactions, ensuring security in online payment systems has become increasingly critical. This project presents a Fraud-Aware E-Wallet System designed to provide secure, efficient, and user-friendly digital payment services while actively detecting and preventing fraudulent activities. The system allows users to perform essential financial operations such as account creation, balance management, money transfers, and transaction history tracking. In addition to these standard features, the system integrates fraud detection mechanisms that monitor user behavior and transaction patterns to identify suspicious activities. Alerts are generated for unusual transactions, such as abnormal transfer amounts or irregular login attempts, enhancing the overall security of the platform. The application is developed using object-oriented programming principles, ensuring modularity, scalability, and maintainability. It also includes features like authentication, transaction validation, and secure data handling to protect user information. This project aims to demonstrate how intelligent monitoring and basic fraud detection techniques can be incorporated into digital payment systems to improve reliability and user trust. The Fraud-Aware E-Wallet System serves as a foundational model for building more advanced and secure financial applications in the future.

INTRODUCTION

In today's digital era, the use of electronic payment systems has significantly increased due to their convenience, speed, and accessibility. E-wallets have become an essential part of daily transactions, enabling users to transfer money, pay bills, and manage finances with ease. However, with the rapid adoption of digital payments, the risk of fraudulent activities such as unauthorized access, identity theft, and suspicious transactions has also grown.

This project, "Fraud-Aware E-Wallet System," aims to address these challenges by combining the functionality of a traditional e-wallet with enhanced security features. The system is designed to provide a safe and reliable platform for users to perform financial transactions while continuously monitoring activities to detect potential fraud.

The application includes features such as user authentication, balance management, secure money transfers, and transaction history tracking. Additionally, it incorporates basic fraud detection mechanisms that identify unusual patterns, such as abnormal transaction amounts or multiple failed login attempts, and alerts the user accordingly.

By integrating security measures with user-friendly design, this project highlights the importance of building trustworthy digital financial systems. It also demonstrates how technology can be used not only to simplify transactions but also to protect users from emerging cyber threats.

LITREATURE REVIEW

The rapid growth of digital payment systems has led to the widespread adoption of e-wallets as a convenient and efficient mode of financial transactions. Various studies have explored the design, implementation, and security aspects of e-wallet systems, highlighting both their advantages and associated risks. Existing research shows that traditional e-wallet systems primarily focus on providing basic functionalities such as fund transfer, bill payments, and transaction tracking. However, these systems often lack advanced fraud detection mechanisms to identify and prevent fraudulent activities in real time. As a result, users remain vulnerable to threats such as unauthorized access, phishing attacks, and identity theft. Several researchers have proposed the integration of security features like user authentication, encryption, and secure transaction protocols to enhance the reliability of digital wallets. Additionally, modern approaches emphasize the use of behavioral analysis and pattern recognition techniques to identify suspicious transactions. These techniques analyze user activity patterns, such as transaction frequency, location, and amount, to detect anomalies. Furthermore, studies have shown that implementing alert systems for unusual activities can significantly improve user awareness and help prevent potential fraud. Notification-based systems and multi-factor authentication have also been identified as effective methods to strengthen security. Despite these advancements, there is still a need for simple, efficient, and cost-effective fraud detection mechanisms that can be easily integrated into basic e-wallet systems. This project addresses this gap by developing a Fraud-Aware E-Wallet System that combines essential wallet functionalities with basic fraud detection and alert features, ensuring improved security, reliability, and user trust.

SYSTEM ANALYSIS

Problem Definition

With the increasing use of digital payment systems, users rely heavily on e-wallets for daily financial transactions such as money transfers, bill payments, and online purchases. However, many existing systems lack effective fraud detection and prevention mechanisms, making them vulnerable to threats like unauthorized access, phishing, and suspicious transactions. These issues can lead to financial

losses and reduced user trust. Therefore, there is a need to develop a system that not only provides basic e-wallet functionalities but also ensures enhanced security by detecting and alerting users about potential fraudulent activities. The proposed Fraud-Aware E-Wallet System aims to address this problem by integrating security features with transaction monitoring.

1. Feasibility Study. a) Technical

The system is developed using industry-standard technologies. The frontend uses HTML, CSS, and JavaScript. The backend API is built with Node.js and Express.js following the MVC (Model-View-Controller) architecture pattern. MySQL 8.0 serves as the relational database management system, providing advanced features such as stored procedures, triggers, views, and ACID-compliant transactions. Firebase Authentication handles secure user login. The ML fraud detection model is built using Python's scikit-learn library and served via a Flask REST API. All tools used are open-source and freely available, making the project technically feasible without any licensing costs.

b) Operational

The system is designed to be user-friendly and easy to operate, even for users with minimal technical knowledge. Features like simple navigation, clear prompts, and alert messages ensure smooth interaction. The system can be efficiently used for managing transactions while also providing warnings for suspicious activities, making it practical for real-world usage.

2. Requirements

a) Functional Requirements

These define what the system should do:

- User registration and login (authentication system) □
- Add and manage wallet balance □ Perform secure money transfers
- View transaction history
- Detect suspicious transactions based on predefined conditions
- Generate alerts for unusual activities
- Store and manage user data securely

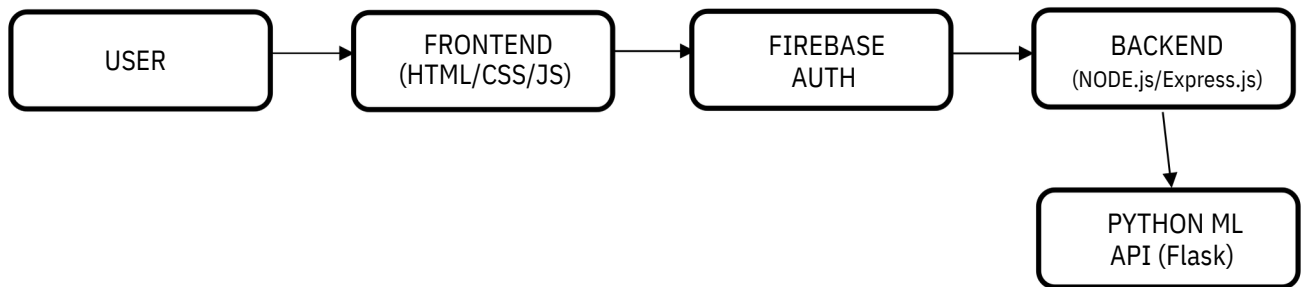
b) Non-Functional Requirements

These define how the system should perform:

- Security: Protection of user data and prevention of unauthorized access
- Performance: Quick processing of transactions and responses
- Reliability: Accurate transaction handling without errors
- Usability: Simple and easy-to-use interface
- Scalability: Ability to handle more users and features in future
- Maintainability: Easy to update and modify the system

SYSTEM DESIGN

Architecture Diagram



The system follows a 4 – tier architecture:

□ Presentation Layer (FRONTEND)

- o Built using HTML, CSS, and JavaScript interactive UI with pages for Dashboard, Send Money, Transactions, Fraud Alerts, and Profile. Communicates with the backend via REST API calls.

□ Authentication Layer (FIREBASE)

- o Handles user authentication via Google Sign-In and Email/Password. Issues JWT tokens that the backend verifies using the Firebase Admin SDK. Users must exist in the MySQL USER table to access the wallet.Database Layer (DATABASE)

□Application Layer (BACKEND)

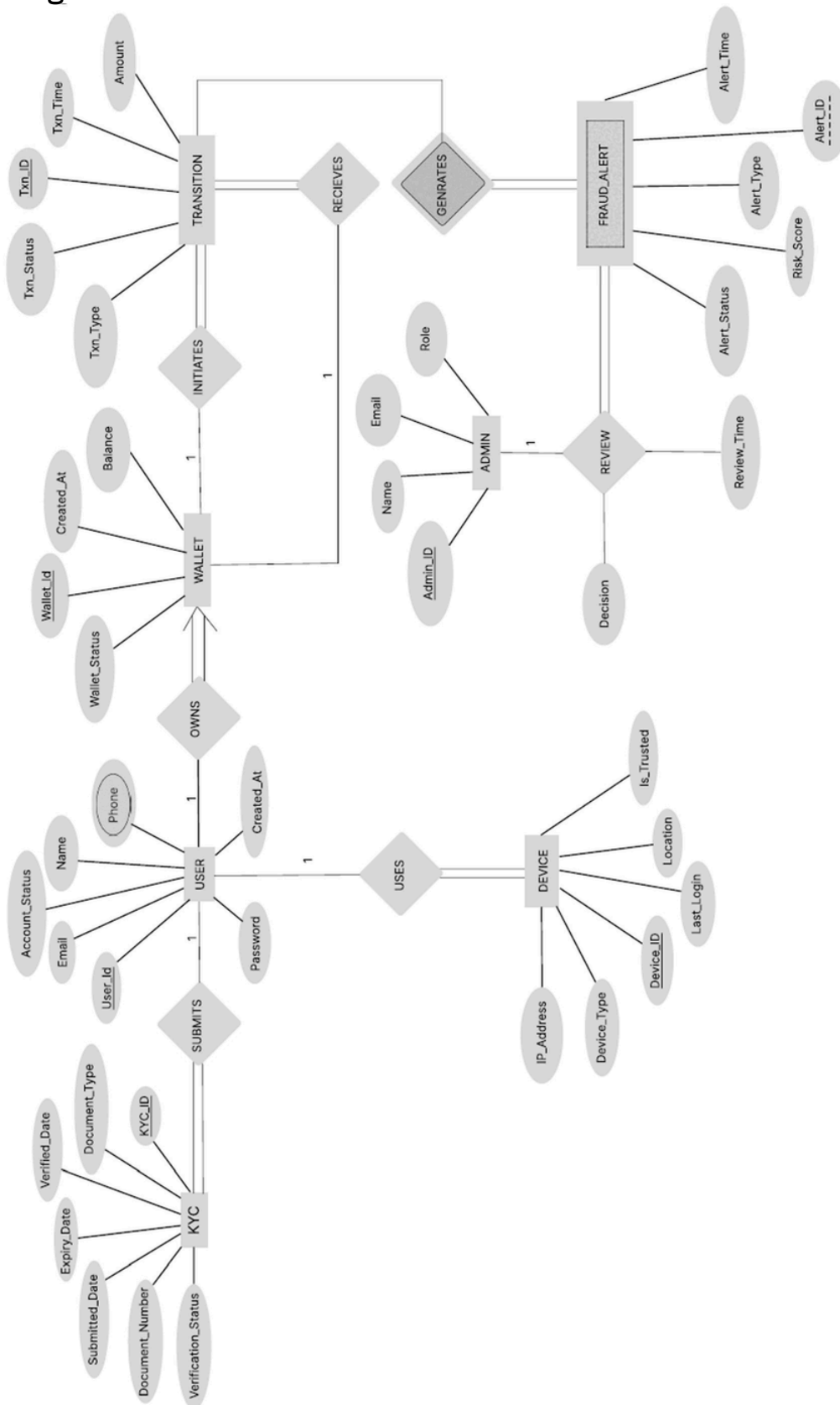
- o Node.js with Express.js implementing MVC architecture. Routes API requests to controllers that execute SQL queries against MySQL. Computes 15 ML features from transaction history and calls the Python ML API for fraud scoring.

□DataLayer (MySQL + ML)

- o MySQL 8.0 persistent data across 10 tables. Stored procedures handle ACID-compliant fund transfers. Triggers auto-generate fraud alerts. A Python Flask API serves the trained Gradient Boosting model for real-time fraud prediction.

DATABASE DESIGN

ER Diagram



RELATIONAL SCHEMA :

1. USER(User_ID, firebase_uid, Name, Email, Phone, Account_Status, Created_At)
2. WALLET(Wallet_ID, Balance, Wallet_Status, Created_Date, User_ID)
3. DEVICE(Device_ID, Device_Type, IP_Address, Location, Is_Trusted, Last_Login, User_ID)
4. KYC(KYC_ID, Document_Type, Document_Number, Verification_Status, Submitted_Date, Verified_Date, Expiry_Date, User_ID)
5. TRANSACTION(Txn_ID, Amount, Txn_Type, Txn_Time, Txn_Status, Sender_Wallet_ID, Receiver_Wallet_ID)
6. FRAUD_ALERT(Alert_ID, Alert_Type, Risk_Score, Alert_Time, Alert_Status, Txn_ID)
7. ADMIN(Admin_ID, Name, Email, Role)
8. REVIEW(Alert_ID, Admin_ID, Decision, Review_Time)
9. AUDIT_LOG(Log_ID, Table_Name, Record_ID, Action, Old_Value, New_Value, Changed_By, Changed_At)
10. TRANSACTION_FEATURES(Txn_ID, sender_txn_count_1h, sender_txn_count_24h, sender_avg_amount, amount_deviation, is_new_receiver, hour_of_day, is_night_transaction, device_is_trusted, sender_account_age_days)

TABLES :

1. USER Table :

User_ID	Name	Email	Phone	Password	Account_Status	Created_At
U001	Arjun Rao	arjun@mail.com	9876543210	pass1	Active	2024-01-01
U002	Sneha Mehta	sneha@mail.com	9123456780	pass2	Active	2024-01-02
U003	Rahul Verma	rahul@mail.com	9012345678	pass3	Suspended	2024-01-03
U004	Priya Singh	priya@mail.com	9988776655	pass4	Active	2024-01-04
U005	Amit Shah	amit@mail.com	9898989898	pass5	Active	2024-01-05
U006	Neha Kapoor	neha@mail.com	9777665544	pass6	Active	2024-01-06
U007	Karan Malhotra	karan@mail.com	9666554433	pass7	Blocked	2024-01-07
U008	Isha Jain	isha@mail.com	9555443322	pass8	Active	2024-01-08
U009	Vikram Nair	vikram@mail.com	9444332211	pass9	Active	2024-01-09
U010	Anjali Das	anjali@mail.com	9333221100	pass10	Active	2024-01-10

2. WALLET Table :

Wallet_ID	Balance	Wallet_Status	Created_Date	User_ID
W101	5000	Active	2024-01-01	U001
W102	3000	Active	2024-01-02	U002
W103	2000	Frozen	2024-01-03	U003
W104	4500	Active	2024-01-04	U004
W105	7000	Active	2024-01-05	U005
W106	1500	Active	2024-01-06	U006
W107	900	Frozen	2024-01-07	U007
W108	6000	Active	2024-01-08	U008
W109	8000	Active	2024-01-09	U009
W110	2500	Active	2024-01-10	U010

3. DEVICE Table:

Device_ID	Device_Type	IP_Address	Location	Is_Trusted	Last_Login	User_ID
D01	Mobile	192.168.1.1	Delhi	TRUE	2024-02-01	U001
D02	Laptop	192.168.1.2	Mumbai	TRUE	2024-02-02	U002
D03	Tablet	192.168.1.3	Pune	FALSE	2024-02-03	U003
D04	Mobile	192.168.1.4	Chennai	TRUE	2024-02-04	U004
D05	Laptop	192.168.1.5	Kolkata	TRUE	2024-02-05	U005
D06	Mobile	192.168.1.6	Delhi	FALSE	2024-02-06	U006
D07	Tablet	192.168.1.7	Mumbai	TRUE	2024-02-07	U007
D08	Mobile	192.168.1.8	Pune	TRUE	2024-02-08	U008
D09	Laptop	192.168.1.9	Chennai	TRUE	2024-02-09	U009
D10	Mobile	192.168.1.10	Kolkata	TRUE	2024-02-10	U010
		0				

4. KYC Table:

KYC_ID	Document_Type	Document_Number	Verification_Status	Submitted_Date	Verified_Date	Expiry_Date	User_ID
K01	Aadhar	PAN001	Verified	2024-01-01	2024-01-02	2030-01-01	U001
K02	PAN	AAD002	Pending	2024-01-01	2024-01-03	2030-01-01	U002
K03	Passport	PAN003	Verified	02	NULL	2030-01-01	U003
K04	PAN	PAS004	Verified	2024-01-05	2024-01-05	2030-01-01	U004
K05	Aadhar	PAN005	Verified	03	2024-01-06	2030-01-01	U005
K06	PAN	AAD006	Rejected	2024-01-07	2024-01-07	2030-01-01	U006
K07	Passport	PAN007	Verified	04	NULL	2030-01-01	U007
K08	PAN	PAS008	Verified	2024-01-09	2024-01-09	2030-01-01	U008
K09	Aadhar	PAN009	Verified	05	2024-01-10	2030-01-01	U009
K10		AAD010		2024-01-11	2024-01-11	2030-01-01	U010

06

2024-01-

07

2024-01-

5. TRANSACTION Table:

Txn_ID	Amount	Txn_Type	Txn_Time	Txn_Status	Sender_Wallet_ID	Receiver_Wallet_ID
T01	500	Transfer	2024-02-01	Success	W101	W102
T02	1200	Transfer	2024-02-02	Success	W102	W103
T03	700	Transfer	2024-02-03	Failed	W104	W105
T04	2000	Transfer	2024-02-04	Success	W105	W106
T05	450	Transfer	2024-02-05	Success	W106	W107
T06	3000	Transfer	2024-02-06	Blocked	W108	W109
T07	800	Transfer	2024-02-07	Success	W109	W110
T08	1500	Transfer	2024-02-08	Success	W110	W101
T09	600	Transfer	2024-02-09	Success	W102	W104
T10	5000	Transfer	2024-02-10	Success	W105	W108

6. FRAUD_ALERT Table :

Alert_ID	Alert_Type	Risk_Score	Alert_Time	Alert_Status	Txn_ID
A01 A02	HighAmount	85 75 90	2024-02-01	Reviewed	T01 T02
A03 A04	SuspiciousIP	80 70 95	02-02	Reviewed	T03 T04
A05 A06	HighFrequency	65 60 88	2024-02-03	Open	T05 T06
A07 A08	HighAmount	92	2024-02-04	Reviewed	T07 T08
A09 A10	DeviceRisk		05 2024-02-05	Closed	T09 T10
	HighAmount		2024-02-06	Reviewed	
	SuspiciousIP		2024-02-07	Closed	
	DeviceRisk		09 2024-02-09	Closed	
	HighAmount		2024-02-10	Reviewed	
	HighAmount		07 2024-02-07	Open	

7. ADMIN Table :

Admin_ID	Name	Email	Role
AD01	Ravi	ravi@wallet.com	FraudOfficer
AD02	Meera	meera@wallet.com	FraudOfficer
AD03	Sohan	sohan@wallet.com	Auditor
AD04	Kavya	kavya@wallet.com	FraudOfficer
AD05	Rohan	rohan@wallet.com	Auditor
AD06	Ankit	ankit@wallet.com	FraudOfficer
AD07	Nidhi	nidhi@wallet.com	Auditor
AD08	Varun	varun@wallet.com	FraudOfficer
AD09	Simran	simran@wallet.com	Auditor
AD10	Ajay	ajay@wallet.com	FraudOfficer

8. REVIEW Table :

Alert_ID	Admin_ID	Decision	Review_Time
A01 A02	AD01	Approved	2024-02-01
A03 A04	AD02	Approved	2024-02-02
A05 A06	AD03	Pending	2024-02-03
A07 A08	AD04	Approved	2024-02-04
A09 A10	AD05	Closed	2024-02-05
	AD06	Approved	2024-02-06
	AD07	Closed	2024-02-07
	AD08	Closed	2024-02-08
	AD09	Approved	2024-02-09
	AD10	Pending	2024-02-10

9. AUDIT_LOG Table :

Log_ID	Table_Name	Record_ID	Action	Old_Value	Ne w_Value	Changed_By	Changed_At
1	WALLET	W101	UPDATE	{"Balance": 5000.00, "Status": "Active"}	{"Balance": 4500.00, "Status": "Active"}	SYST E M	2024-02-01 00:00:00
2	WALLET	W102	UPDATE	{"Balance": 3000.00, "Status": "Active"}	{"Balance": 3500.00, "Status": "Active"}	SYST E M	2024-02-01 00:00:00
3	WALLET	W102	UPDATE	{"Balance": 3500.00, "Status": "Active"}	{"Balance": 2300.00, "Status": "Active"}	SYST E M	2024-02-02 00:00:00
4	WALLET	W103	UPDATE	{"Balance": 2000.00, "Status": "Frozen"}	{"Balance": 3200.00, "Status": "Frozen"}	SYST E M	2024-02-02 00:00:00
5	WALLET	W105	UPDATE	{"Balance": 7000.00, "Status": "Active"}	{"Balance": 5000.00, "Status": "Active"}	SYST E M	2024-02-04 00:00:00
6	WALLET	W106	UPDATE	{"Balance": 1500.00, "Status": "Active"}	{"Balance": 3500.00, "Status": "Active"}	SYST E M	2024-02-04 00:00:00
7	WALLET	W109	UPDATE	{"Balance": 8000.00, "Status": "Active"}	{"Balance": 7200.00, "Status": "Active"}	SYST E M	2024-02-07 00:00:00
8	WALLET	W110	UPDATE	{"Balance": 2500.00, "Status": "Active"}	{"Balance": 3300.00, "Status": "Active"}	SYST E M	2024-02-07 00:00:00
9	WALLET	W110	UPDATE	{"Balance": 3300.00, "Status": "Active"}	{"Balance": 1800.00, "Status": "Active"}	SYST E M	2024-02-08 00:00:00
10	WALLET	W101	UPDATE	{"Balance": 4500.00, "Status": "Active"}	{"Balance": 6000.00, "Status": "Active"}	SYST E M	2024-02-08 00:00:00

10. TRANSACTION_FEATURES Table :

Txn_ID	txn_count_1h	txn_count_24h	avg_amount	amount_dev	is_new_recv	hour	is_night	device_trusted	acct_age_days
T0	0	1	50 0. 00	0.00	1	1	0	1	36
1	0	2	85 0. 00	0.82	1	4	0	1	5
T0	0	1	70 0. 00	0.00	0	1	0	1	36
2	0	1	20 00.	0.00	1	0	0	1	6
T0	1	3	00 72 5.	-0.5	1	1	0	0	36
3	0	1	00 30	5	1	6	1	1	7
T0	0	2	00. 00	0.00	0	1	0	1	36
4	0	1	80 0. 00	0.00	1	1	0	1	8
T0	1	4	15 00.	0.00	0	9	0	1	36
5	0	2	00 76 6.	-0.2	1	2	0	1	9
T0			67 35	9		1			37
6			00. 00	0.86		5			0
T0						2			37
7						0			1
T0						1			37

KEYS

Primary Keys

Primary keys uniquely identify each record in a table.

- USER → user_id
- WALLET → wallet_id
- DEVICE → device_id
- KYC → kyc_id
- TRANSACTION → txn_id
- FRAUD_ALERT → alert_id
- ADMIN → admin_id
- REVIEW → (alert_id, admin_id) (composite key)
- AUDIT_LOG → Log_ID (AUTO_INCREMENT)
- TRANSACTION_FEATURES → Txn_ID

Foreign Keys

Foreign keys are used to establish relationships between tables.

- WALLET.user_id → references USER.user_id
- DEVICE.user_id → references USER.user_id
- KYC.user_id → references USER.user_id
- TRANSACTION.sender_wallet_id → references
WALLET.wallet_id
- TRANSACTION.receiver_wallet_id → references
WALLET.wallet_id
- FRAUD_ALERT.txn_id → references TRANSACTION.txn_id
- REVIEW.alert_id → references FRAUD_ALERT.alert_id
- REVIEW.admin_id → references ADMIN.admin_id
- TRANSACTION_FEATURES.Txn_ID → references
TRANSACTION.Txn_ID

FUNCTIONAL DEPENDENCIES

□USER

User_ID → Name, Email, Phone, Password,
Account_Status, Created_At

□WALLET

Wallet_ID → Balance, Wallet_Status, Created_Date,
User_ID

□DEVICE

Device_ID → Device_Type, IP_Address, Location,
Is_Trusted, Last_Login, User_ID

□KYC

KYC_ID → Document_Type, Document_Number,
Verification_Status, Submitted_Date, Verified_Date,
Expiry_Date, User_ID

□ TRANSACTION

Txn_ID → Amount, Txn_Type, Txn_Time, Txn_Status,
Sender_Wallet_ID, Receiver_Wallet_ID

□ FRAUD_ALERT

Alert_ID → Alert_Type, Risk_Score, Alert_Time,
Alert_Status, Txn_ID

□ADMIN

Admin_ID → Name, Email, Role

□REVIEW

(Alert_ID, Admin_ID) → Decision, Review_Time

□AUDIT_LOG

Log_ID → Table_Name, Record_ID, Action, Old_Value,
New_Value, Changed_By, Changed_At

□ TRANSACTION_FEATURES

Txn_ID → sender_txn_count_1h, sender_txn_count_24h,
sender_avg_amount, amount_deviation, is_new_receiver,
hour_of_day, is_night_transaction, device_is_trusted,
sender_account_age_days

IMPLEMENTATION

Frontend Implementation:

Technology: HTML5, CSS3, JavaScript (Vanilla)

File Structure:

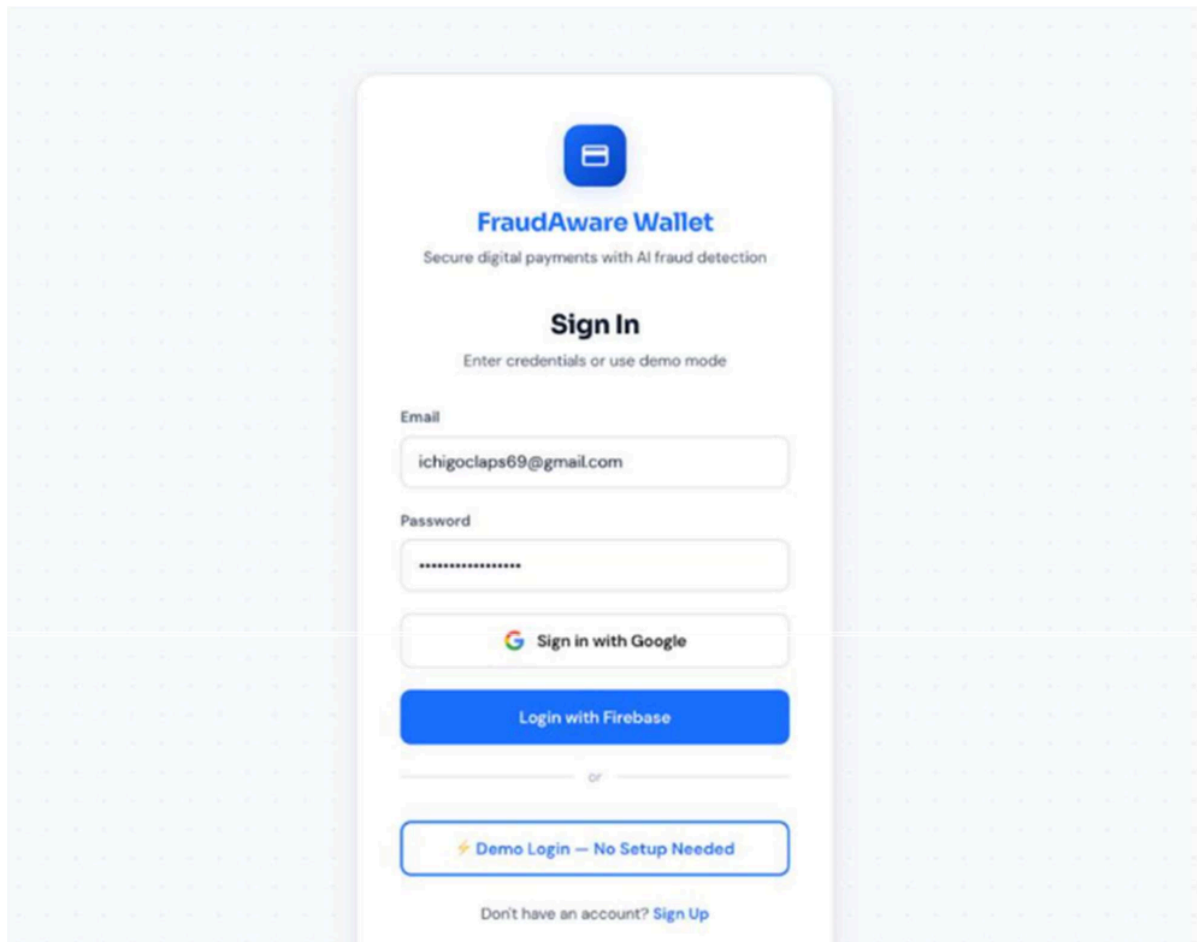
- ☐ index.html — Main HTML structure (single-page application)
- ☐ css/styles.css — Design tokens, layout, navigation, sidebar
- ☐ css/components.css — Reusable UI components (buttons, cards, forms, tables)
- ☐ css/pages.css — Page-specific styles (dashboard, send money, fraud alerts)
- ☐ js/config.js — Firebase configuration and API base URL
- ☐ js/api.js — Authentication logic and HTTP client
- ☐ js/pages.js — Data loading and rendering for each page
- ☐ js/app.js — Navigation, toast notifications, modal handlers

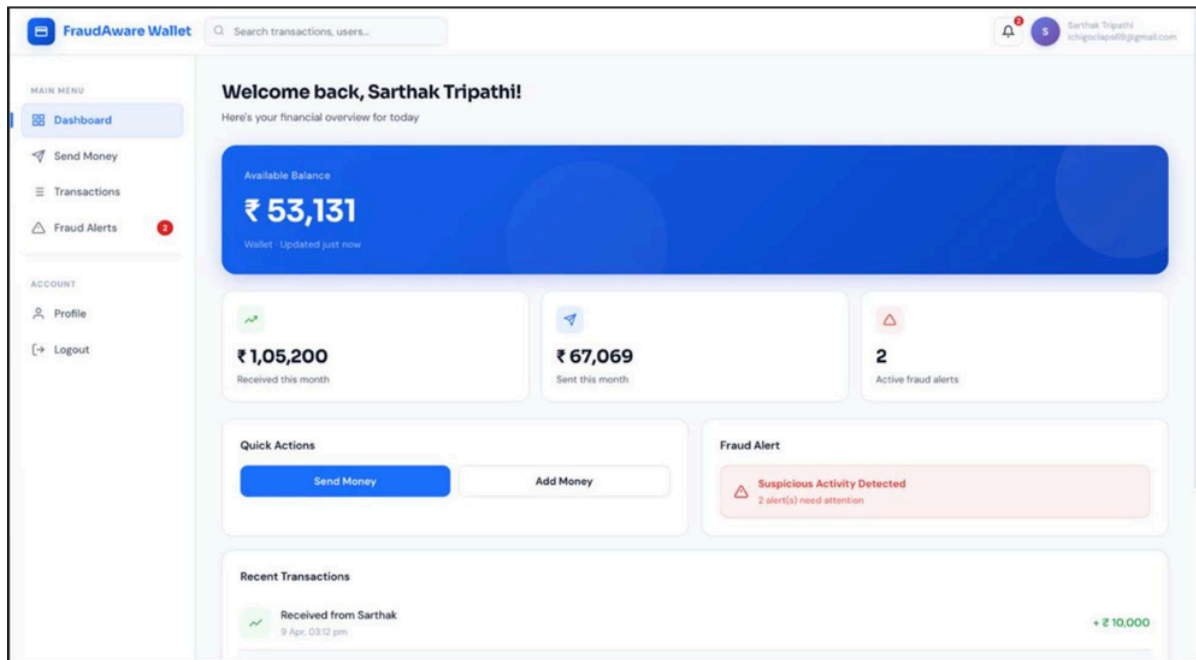
Key Features:

- ☐ Single-page application with sidebar navigation (no page reloads)
- ☐ Responsive design with CSS custom properties for consistent theming
- ☐ Real-time fraud score display with color-coded risk levels (green/amber/red)
- ☐ Dynamic data fetching from backend API on every page navigation
- ☐ Firebase Google Sign-In and Email/Password authentication integration

Pages Developed:

- ❑ Login Page — Firebase authentication with Google Sign-In, Email/Password login, Demo mode, and Sign Up with auto-registration
- ❑ Dashboard — Wallet balance, monthly statistics (sent/received), active fraud alerts count, and recent transactions
- ❑ Send Money — Recipient email input, amount, optional note, real-time fraud score result with ML feature details
- ❑ Transaction History — Filterable table (All/Sent/Received/Flagged) with fraud scores and status chips
- ❑ Fraud Alerts — Auto-generated alert cards with risk meters, transaction details, and Block/Mark Safe actions
- ❑ Profile — User information display with wallet statistics





Backend Implementation :

Technology: Node.js 18+ with Express.js framework

Architecture: MVC (Model-View-Controller) pattern

File Structure:

- ❑ server.js — Express application entry point, middleware setup, route mounting
- ❑ config/db.js — MySQL connection pool (10 concurrent connections)
- ❑ config/firebase.js — Firebase Admin SDK initialization using service account key
- ❑ middleware/auth.js — JWT token verification and demo mode authentication
- ❑ routes/api.js — All REST API route definitions
- ❑ controllers/walletController.js — Wallet balance and statistics logic
- ❑ controllers/transactionController.js — Send money with 15-feature ML computation
- ❑ controllers/fraudController.js — Fraud alerts listing and resolution

API Endpoints Implemented:

Method	Endpoint	Purpose	MySQL Operation
POST	/api/register	Auto-register new user	INSERT into USER + WALLET
POST	/api/check-email	Verify email exists	SELECT from USER
GET	/api/wallet	Get wallet info	SELECT from vw_user_wallet (view)
GET	/api/wallet/stat	Dashboard statistics	3 aggregate queries (SUM, COUNT)
POST	s /api/add-money	Add funds to wallet	CALL sp_add_money (stored procedure)
POST	/api/send-money	Transfer with fraud check	7 SQL queries + CALL sp_send_money
GET	/api/transactions	Transaction history	Multi-table JOIN with filters
GET	/api/fraud-alerts	List fraud alerts	JOIN across 4 tables
POST	/api/fraud-check	Manual fraud score check	ML prediction only
POST	/api/fraud-alerts/:id/resolve	Block or approve alert	UPDATE on FRAUD_ALERT + TRANSACTION

Connection Pooling:

The backend uses mysql2 library with a connection pool of 10 connections. This reuses existing database connections instead of creating a new TCP connection for every request, reducing latency from ~50ms to ~1ms per quer

Database Implementation (MySQL 8.0) :

Table USER	Columns	Purpose
WALLET	7	Registered users with Firebase UID
TRANSACTION	5	One wallet per user with balance
FRAUD_ALERT	11	Every money movement with fraud score
DEVICE	6	Auto-generated alerts for suspicious transactions
KYC	7	User device tracking for ML features
ADMIN REVIEW	8	Know Your Customer documents
AUDIT_LOG	4	Fraud officers and auditors
TRANSACTION_FEATURES	4	Admin decisions on fraud alerts
	8	Immutable record of all balance changes (NEW)
Stored Procedures:	10	ML feature values per transaction (NEW)

sp_send_money — ACID-compliant fund transfer:

1. START TRANSACTION — begins atomic operation
2. SELECT Balance, Wallet_Status FROM WALLET WHERE Wallet_ID = ? FOR UPDATE — locks sender row (pessimistic locking)
3. SELECT Wallet_Status FROM WALLET WHERE Wallet_ID = ? FOR UPDATE — locks receiver row
4. Validates: sufficient balance, wallet not frozen, fraud score threshold
5. If fraud score $\geq 90 \rightarrow$ INSERT transaction as 'Blocked', do not move money
6. Otherwise $\rightarrow$ UPDATE WALLET SET Balance = Balance - amount (debit sender)
7. UPDATE WALLET SET Balance = Balance + amount (credit receiver)
8. INSERT transaction record with fraud score
9. COMMIT — all changes permanent
10. If any error $\rightarrow$ ROLLBACK — nothing changes (atomicity)

Trigger	Event	Action
trg_fraud_on_insert	AFTER INSERT on TRANSACTION	Creates FRAUD_ALERT when fraud_score ≥ 60
trg_auto_fraud_flag_insert	AFTER INSERT on TRANSACTION	Backup fraud alert creation
trg_wallet_audit	AFTER UPDATE on WALLET	Logs old/new balance to AUDIT_LOG as JSON
trg_block_frozen_wallet	BEFORE INSERT on TRANSACTION	Sets Txn_Status = 'Blocked' if sender wallet is frozen

View	Tables Joined	Purpose
vw_fraud_summary	FRAUD_ALERT + TRANSACTION + WALLET + USER (x2)	5-table JOIN for fraud dashboard
vw_user_wallet	USER + WALLET	Used by auth middleware on every request
vw_transaction_history	TRANSACTION + WALLET (x2) + USER (x2)	Full transaction details with names

Index	Table.Column	Why
idx_firebase_uid	USER.Firebase_UID	Used for lookup on every API request
idx_txn_sender	TRANSACTION.Sender_Wallet_ID	JOIN for transaction details dashboard
idx_txn_receiver	TRANSACTION.Receiver_Wallet_ID	JOIN performance on dashboard
idx_txn_time	TRANSACTION.Txn_Time	Date range filter queries
idx_txn_flagged	TRANSACTION.is_flagged	Flagged filter on transactions page
idx_alert_txn	FRAUD_ALERT.Txn_ID	JOIN between alerts and transactions

ML Fraud Detection Implementation

Technology: Python 3.14, scikit-learn, Flask

File Structure:

- train_model.py – Training script (generates data, trains model, saves .pkl file)
- fraud_api.py – Flask REST API serving predictions on port 5001
- model/fraud_model.pkl – Serialized trained model
- requirements.txt – Python dependencies

Algorithm: Gradient Boosted Decision Trees

- 300 trees with max depth 7
- Learning rate: 0.08
- Subsample: 0.8 (stochastic gradient boosting)
- Trained on 150,000 synthetic transactions (120,000 legitimate + 30,000 fraudulent)

7 Fraud Patterns in Training Data

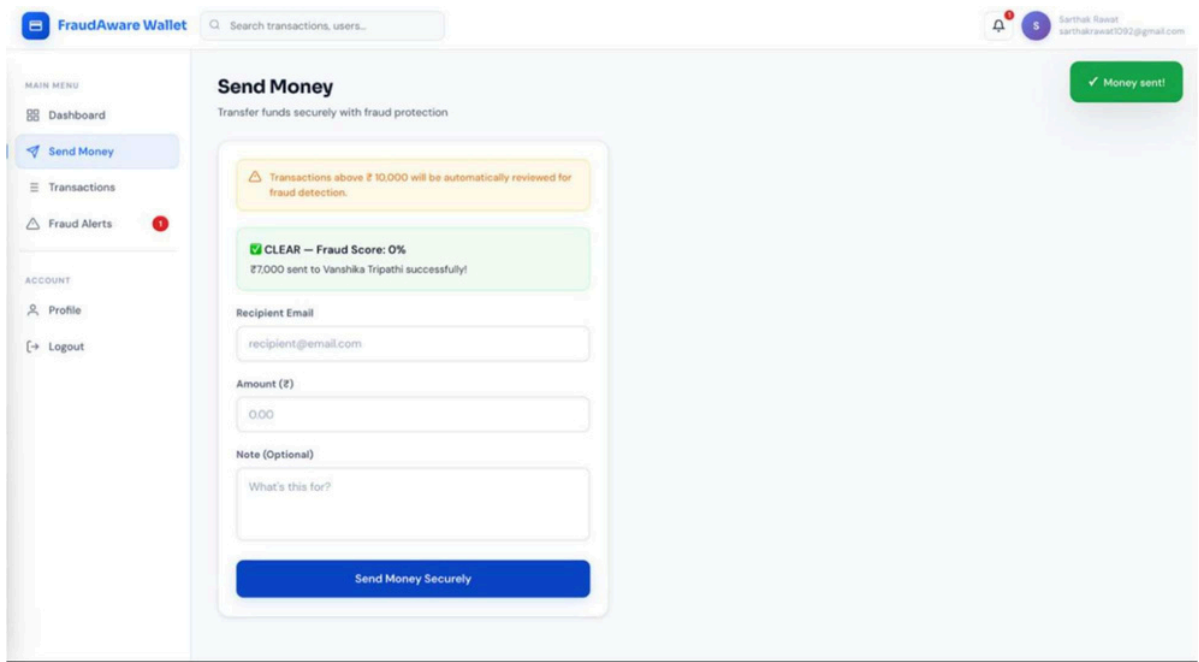
Pattern	Characteristics	Example
Account Takeover	Large amount, night time, untrusted device, new receiver	Stolen credentials draining wallet
Card Testing	Tiny amounts (₹1–₹100), 10–20 txns per hour	Checking if stolen account works Burst transfers before
Velocity Attack	Medium amounts, 6–12 txns per hour	detection Structuring to avoid
Smurfing	Amounts just under limits (₹9,999), many recipients	detection Victim tricked into
Social Engineering	Large amount, new receiver, daytime, own device	sending money Fraudster creates and drains
New Account Fraud	Account < 5 days old, large transfers	Dormant account compromised
Sleeper Fraud	Old account, sudden large unusual transfer at night	

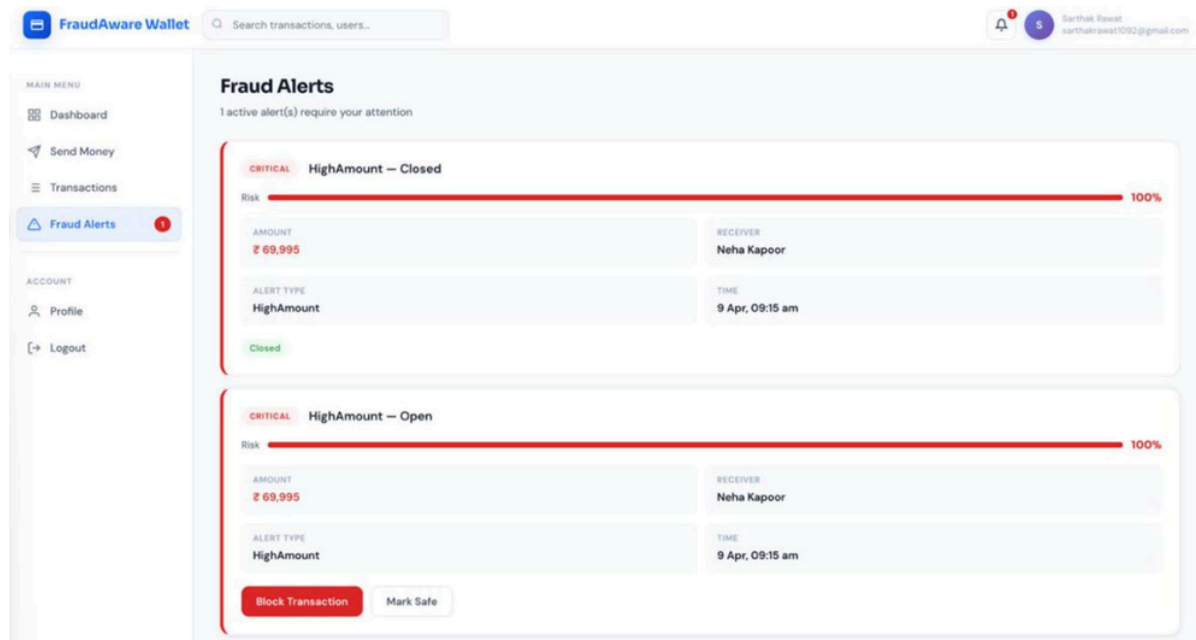
15 Features Computed from MySQL

#	Feature	amount	SQL Query	Purpose
1	sender_txn_count_1h		From request COUNT(*) WHERE	Transaction size
2			Txn_Time >= NOW() - 1 HOUR	Burst detection Daily
3	sender_txn_count_24h		COUNT(*) WHERE Txn_Time >= NOW() - 24 HOURS	anomaly User baseline Statistical
4	sender_avg_amount amount_deviation		WHERE Sender_Wallet_ID = ?	anomaly First-time
5	is_new_receiver hour_of_day		(amount - AVG) / STDDEV COUNT(*) WHERE	recipient Time pattern
6	is_night_transaction device_is_trusted		sender→receiver = 0 HOUR(NOW())	Night flag Device check
7			hour >= 23 OR hour <= 4	Relative
8	amount_to_avg_ratio		MAX(Is_Trusted) FROM	anomaly
9	txn_velocity_score receiver_risk_score		DEVICE amount / AVG(Amount)	Speed metric Composite
11	amount_round_flag		count_1h × 3 + count_24h × 0.5	risk Round
12	time_since_last_txn		is_new × (amount + night factors)	number flag
13			amount % 1000 == 0	
14				
15			TIMESTAMPDIFF(MINUTE, Rapid MAX(Txn_Time), NOW())	detection

Integration with Backend:

1. User clicks "Send Money" on frontend
2. Backend receives request and runs 7 SQL queries to compute all 15 features from MySQL
3. Backend sends features as JSON to Flask API: POST `http://localhost:5001/predict`
4. ML model returns: `{ fraud_score: 85.2, is_fraud: true, risk_level: "CRITICAL" }`
5. Backend passes fraud score to stored procedure `sp_send_money`
6. If score $\geq 90 \rightarrow$ transaction blocked; if score $\geq 60 \rightarrow$ transaction flagged; else $\rightarrow$ approved
7. MySQL trigger auto-creates `FRAUD_ALERT` for flagged/blocked transactions





Firestore Authentication Implementation

Technology: Firestore Authentication (Google Identity Platform)

Components:

- Firestore Console: Project "Fraud-Aware-Digital-Wallet" with Email/Password and Google Sign-In providers enabled
- Frontend: Firestore JS SDK (v10.7.1) loaded via CDN script tag
- Backend: Firestore Admin SDK (npm package firestore-admin) with service accountkey for server-side token verification

Sign-In Methods Enabled:

Email/Password — user creates account with email and password

Google Sign-In — one-click login using Google account

Auto-Registration Feature:

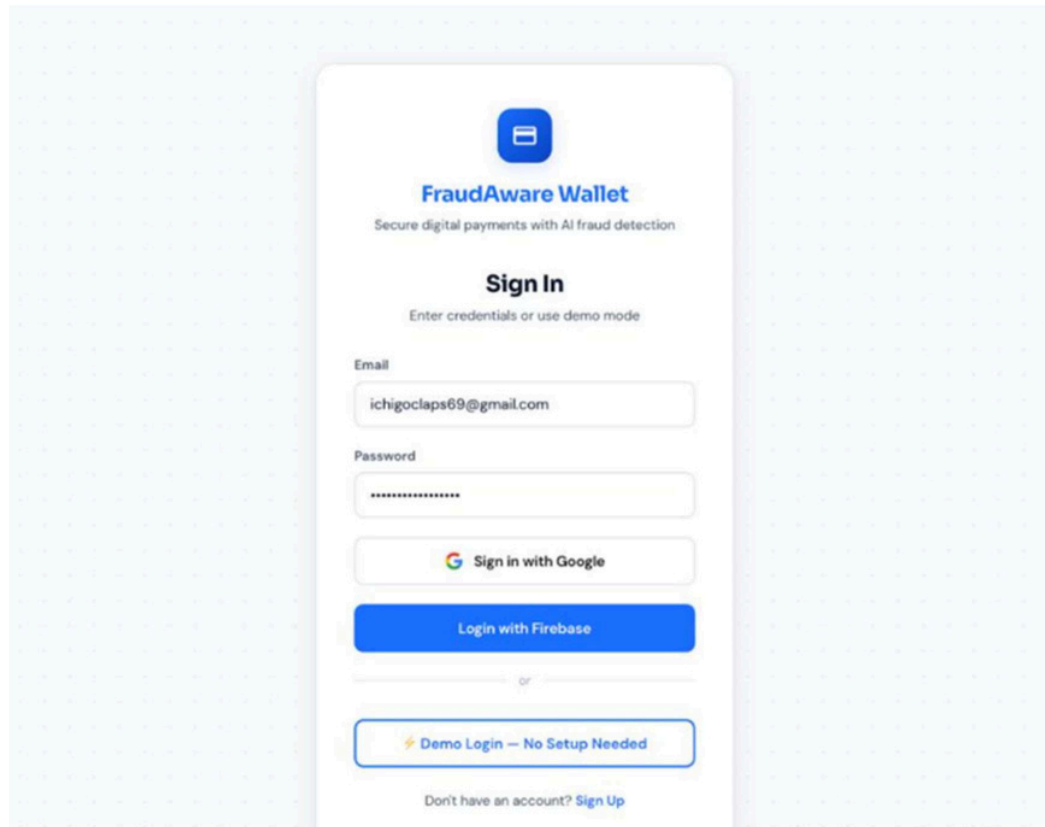
When a new user signs up (either via Email/Password or Google), the backend automatically creates their USER record and WALLET in MySQL with a starting balance of ₹10,000. This eliminates the need for manual SQL insertion by an admin.

New User Sign Up :

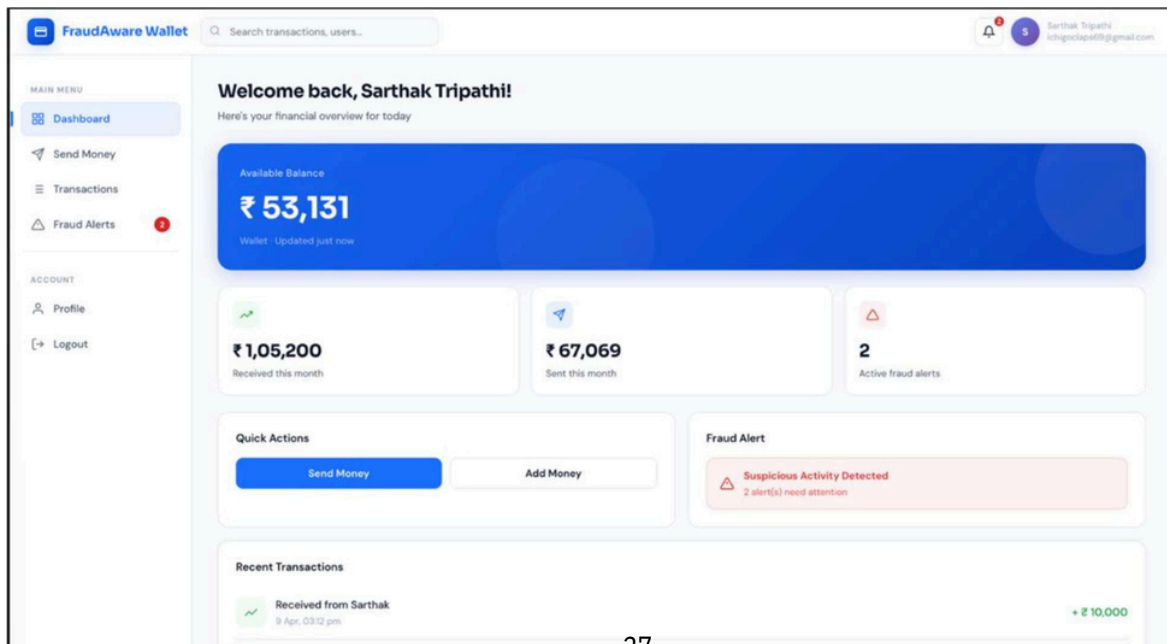
1. → Firebase creates auth account
2. → Backend calls /api/register
3. → INSERT INTO USER (auto-generated User_ID, email, name)
4. → INSERT INTO WALLET (auto-generated Wallet_ID, ₹10,000 balance)
5. → User is logged in with their new wallet

SCREENSHOTS/OUTPUT

1. Login Page :



2. Dashboard :



3. Send Money :

FraudAware Wallet Search transactions, users...

Send Money
Transfer funds securely with fraud protection

Transactions above ₹ 10,000 will be automatically reviewed for fraud detection.

CLEAR — Fraud Score: 0%
₹7,000 sent to Vanshika Tripathi successfully!

Recipient Email
recipient@email.com

Amount (₹)
0.00

Note (Optional)
What's this for?

Send Money Securely

Money sent!

4. Transaction:

FraudAware Wallet Search transactions, users...

Transaction History
All your wallet activity in one place

All Sent Received Flagged

NAME	AMOUNT	TYPE	DATE	FRAUD SCORE	STATUS
Vanshika Tripathi vanshika.tripathi06@gmail.com	₹ 7,000	Sent	12 Apr 2026	0%	✓ Clear
Vanshika Tripathi vanshika.tripathi06@gmail.com	₹ 50	Sent	12 Apr 2026	0.01%	✓ Clear
Vanshika Tripathi vanshika.tripathi06@gmail.com	₹ 1,00,000	Sent	12 Apr 2026	5.45%	✓ Clear
Vanshika Tripathi vanshika.tripathi06@gmail.com	₹ 299	Sent	12 Apr 2026	99.42%	✓ Clear
Sarthak Tripathi ichigoclap09@gmail.com	₹ 10,000	Sent	9 Apr 2026	0%	✓ Clear
Sarthak Tripathi ichigoclap09@gmail.com	₹ 1,000	Sent	9 Apr 2026	0.01%	✓ Clear
Sarthak Tripathi ichigoclap09@gmail.com	₹ 500	Sent	9 Apr 2026	0%	✓ Clear
Neha Kapoor	₹ 65,995	Sent	9 Apr 2026	100%	⚠ Flagged

5. Blocked Transaction:

Search transactions, users...

MAIN MENU

Dashboard

Send Money

Transactions

Fraud Alerts

ACCOUNT

Profile

Logout

Send Money

Transfer funds securely with fraud protection

Transactions above ₹ 10,000 will be automatically reviewed for fraud detection.

BLOCKED — Fraud Score: 100%

Transaction blocked — fraud risk too high

Recipient Email

recipient@email.com

Amount (₹)

0.00

Note (Optional)

What's this for?

Send Money Securely

Search transactions, users...

MAIN MENU

Dashboard

Send Money

Transactions

Fraud Alerts

ACCOUNT

Profile

Logout

Fraud Alerts

2 active alert(s) require your attention

CRITICAL

HighAmount — Reviewed

Risk

100%

AMOUNT

₹ 1,00,000

RECEIVER

Amit Shah

ALERT TYPE

HighAmount

TIME

12 Apr, 11:13 pm

Reviewed

CRITICAL

HighAmount — Open

Risk

100%

AMOUNT

₹ 1,00,000

RECEIVER

Amit Shah

ALERT TYPE

HighAmount

TIME

12 Apr, 11:13 pm

Block Transaction

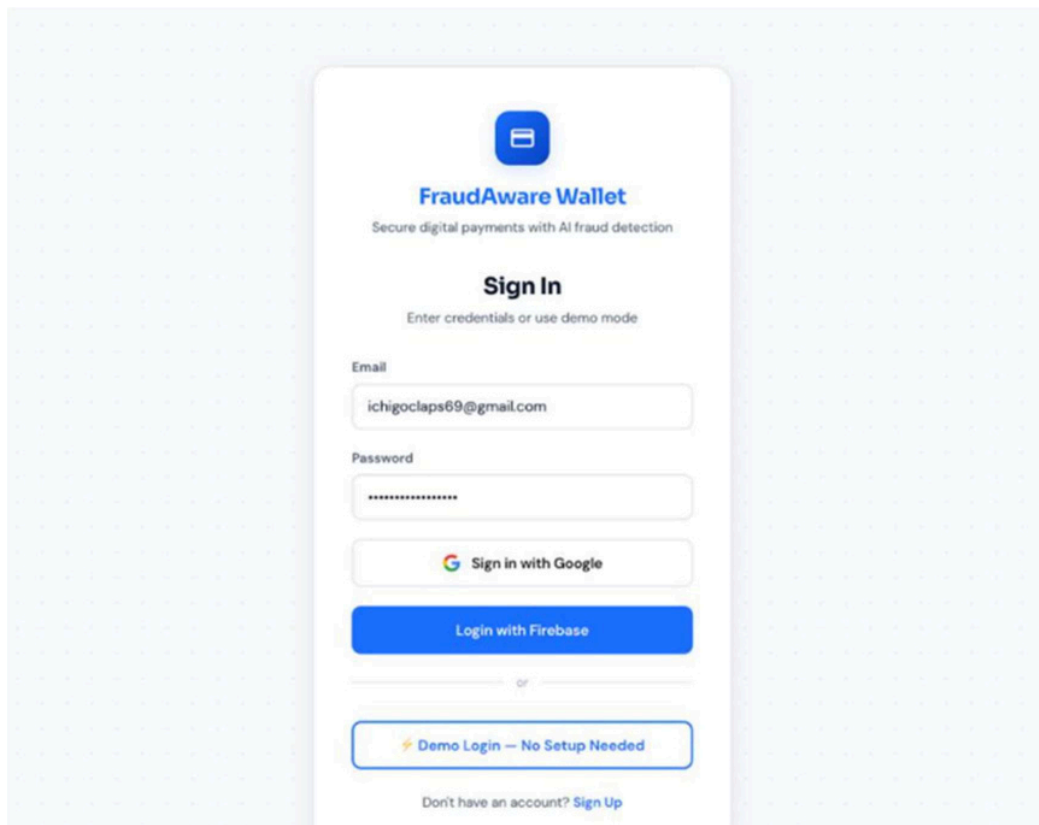
Mark Safe

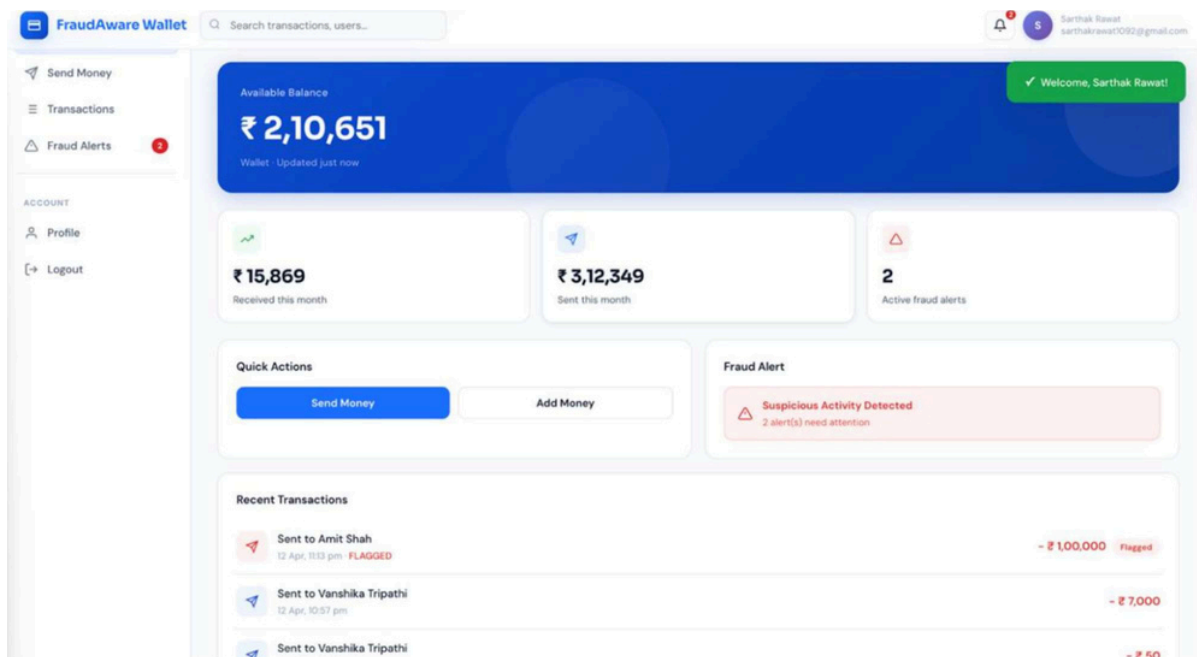
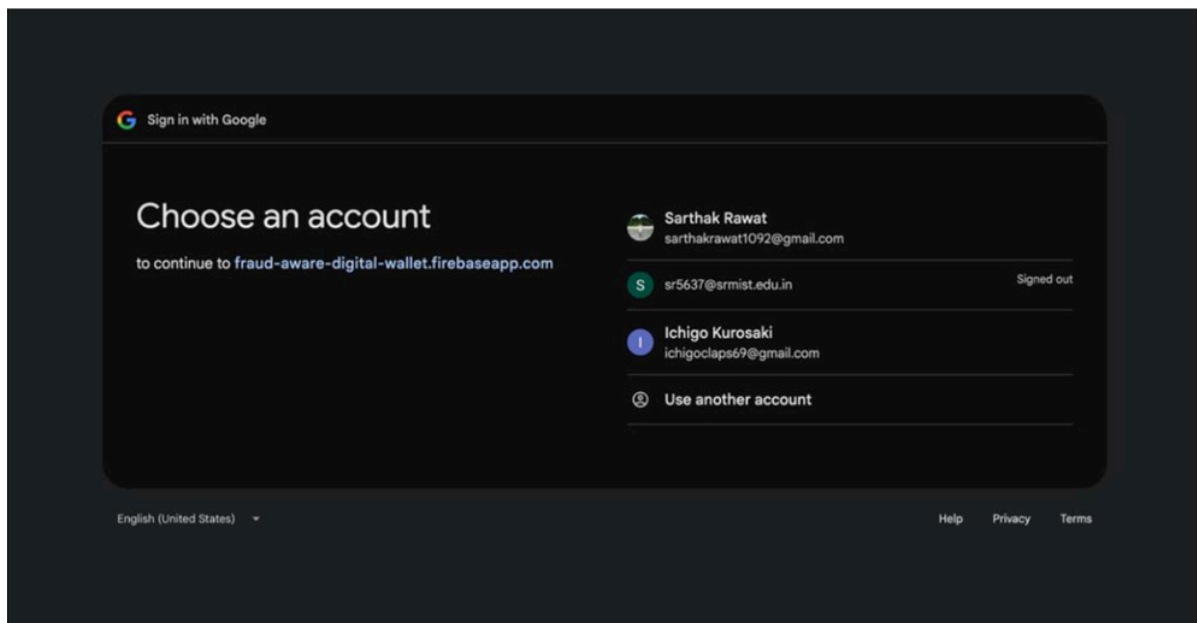
29

TESTING

Test Cases :

Test Case	Input	Expected Output	Actual Output	Status
Normal transaction	₹500 to known receiver	Low fraud score (0-15%), Success	Score: 0%, Status: Success	Pass
High-value suspicious Rapid burst (5 txns)	₹90,000 to new receiver ₹200 × 5 in 1 minute	High fraud score (80%+), Blocked	Score: 100%, Status: Blocked Scores increased	Pass
Frozen wallet send	Send from frozen wallet random@test.com	Increasing fraud scores, eventual blocked	from 0% to 96% Status: Blocked (trigger fired)	Pass
Unregistered email login		Transaction blocked	"Email not registered" error	Pass
Google Sign-In new user			User created with ₹10,000 balance	Pass
Insufficient balance	New Gmail account	Auto-register in MySQL + login	"Insufficient balance" error	Pass
	Send more than balance	Rejected		Pass





FraudAware Wallet

Search transactions, users...

5

Sarthak Rawat

sarthakraut1092@gmail.com

Money sent!

MAIN MENU

Dashboard

Send Money

Transactions

Fraud Alerts

ACCOUNT

Profile

Logout

Send Money

Transfer funds securely with fraud protection

Transactions above ₹ 10,000 will be automatically reviewed for fraud detection.

CLEAR — Fraud Score: 0%

₹7,000 sent to Vanshika Tripathi successfully!

Recipient Email

recipient@email.com

Amount (₹)

0.00

Note (Optional)

What's this for?

Send Money Securely

FraudAware Wallet

Search transactions, users...

5

Sarthak Rawat

sarthakraut1092@gmail.com

MAIN MENU

Dashboard

Send Money

Transactions

Fraud Alerts

ACCOUNT

Profile

Logout

Transaction History

All your wallet activity in one place

All

Sent

Received

Flagged

NAME	AMOUNT	TYPE	DATE	FRAUD SCORE	STATUS
Vanshika Tripathi vanshika.tripathi06@gmail.com	₹ 7,000	Sent	12 Apr 2026	0%	✓ Clear
Vanshika Tripathi vanshika.tripathi06@gmail.com	₹ 50	Sent	12 Apr 2026	0.01%	✓ Clear
Vanshika Tripathi vanshika.tripathi06@gmail.com	₹ 1,00,000	Sent	12 Apr 2026	5.45%	✓ Clear
Vanshika Tripathi vanshika.tripathi06@gmail.com	₹ 299	Sent	12 Apr 2026	59.42%	✓ Clear
Sarthak Tripathi sarthakraut1092@gmail.com	₹ 10,000	Sent	9 Apr 2026	0%	✓ Clear
Sarthak Tripathi sarthakraut1092@gmail.com	₹ 1,000	Sent	9 Apr 2026	0.01%	✓ Clear
Sarthak Tripathi sarthakraut1092@gmail.com	₹ 500	Sent	9 Apr 2026	0%	✓ Clear
Neha Kapoor	₹ 69,995	Sent	9 Apr 2026	100%	⚠ Flagged

Sarathak Rawat
sarthakrawat1092@gmail.com

MAIN MENU

- Dashboard
- Send Money
- Transactions
- Fraud Alerts 1

ACCOUNT

- Profile
- Logout

Fraud Alerts

2 active alert(s) require your attention

CRITICAL

HighAmount – Reviewed

Risk

100%

AMOUNT

₹ 1,00,000

RECEIVER

Amit Shah

ALERT TYPE

HighAmount

TIME

12 Apr, 11:13 pm

Reviewed

CRITICAL

HighAmount – Open

Risk

100%

AMOUNT

₹ 1,00,000

RECEIVER

Amit Shah

ALERT TYPE

HighAmount

TIME

12 Apr, 11:13 pm

Block Transaction

Mark Safe

Sarathak Rawat
sarthakrawat1092@gmail.com

MAIN MENU

- Dashboard
- Send Money
- Transactions
- Fraud Alerts 1

ACCOUNT

- Profile
- Logout

My Profile

Manage your account and security settings

Sarathak
sarthakrawat1092@gmail.com

₹2,10,651

0

0

Balance

Trans

Alerts

Change Password

Enable 2FA

Logout

Personal Information

Full Name

Sarathak

Email Address

sarthakrawat1092@gmail.com

Phone Number

+91 XXXXX XXXXX

Security Settings

Current Password

Enter current password

New Password

Enter new password

Save Changes

✓ Logged out



FraudAware Wallet

Secure digital payments with AI fraud detection

Sign In

Enter credentials or use demo mode

Email

Password

 Sign in with Google

Login with Firebase

or

🔥 Demo Login — No Setup Needed

Don't have an account? [Sign Up](#)

RESULTS AND DISCUSSION

What Was Achieved

The Fraud-Aware E-Wallet System was successfully developed as a full-stack database-driven application combining Firebase Authentication, Node.js backend, MySQL 8.0, and Python ML-based fraud detection. Users can register, login via Email/Password, send and receive money, add funds, and view transaction history — all with real-time fraud scoring on every transaction. The MySQL database implements 10 normalized tables with 4 triggers, 3 views, 2 stored procedures, and ACID-compliant transactions. The ML model trained on 150,000 samples detects 7 real-world attack patterns by analyzing 15 behavioral features computed from live database queries.

Performance and Results

The system performs efficiently across all operations — authentication completes within seconds, fund transfers execute atomically via stored procedures, and the ML model returns fraud scores within milliseconds. B-tree indexes on foreign key columns ensure fast JOIN operations for dashboard and transaction history queries. The fraud detection model accurately differentiates between legitimate and suspicious transactions — the same ₹50,000 amount scores 0% when sent to a known receiver during daytime but 100% when sent to a new receiver at 3AM from a new account. Rapid burst testing confirmed that scores escalate dynamically from 0% to 96% across consecutive transactions as the `sender_txn_count_1h` feature increases in real-time from MySQL.

Obstacles Encountered During Development

The primary challenges included designing a normalized schema with proper constraints across 10 interrelated tables, implementing pessimistic locking in stored procedures to prevent double-spending during concurrent transfers, and bridging two different technology stacks (Node.js and Python) for real-time ML integration. Early versions of the fraud model produced static scores because features were not computed from live MySQL data — this was resolved by writing 7 separate SQL queries (`COUNT`, `AVG`, `STDDEV`, `DATEDIFF`, `TIMESTAMPDIFF`) that extract real behavioral patterns from the user's actual transaction history, making every fraud score contextually unique.

CONCLUSION & FUTURE SCOPE

Conclusion:

This project successfully implemented a full-stack Fraud-Aware E-Wallet System that integrates Firebase Authentication, MySQL 8.0, Node.js backend, and Python ML-based fraud detection. The system demonstrates practical application of DBMS concepts including normalization (3NF), ACID transactions via stored procedures, trigger-based automation, views for query simplification, and strategic indexing for performance optimization. The ML model trained on 150,000 samples across 7 fraud patterns provides dynamic real-time scoring based on 15 behavioral features computed from MySQL.

Future Scope:

- Integration of real payment gateways (Razorpay/Stripe) for actual fund transfers
- Implementation of Google Sign-In authentication (currently Email/Password)
- Real-time WebSocket notifications for instant fraud alerts
- Admin dashboard for fraud officers to review and manage alerts
- Integration with UPI for Indian payment ecosystem compatibility
- Enhanced ML model with deep learning (LSTM) for sequential pattern detection

REFERENCES

1. MySQL 8.0 Reference Manual — Oracle Corporation —
<https://dev.mysql.com/doc/refman/8.0/en/>
2. ACID Properties in Database Systems
& A. Reuter — ACM Computing Surveys, Vol. 15,
No. 4, 1983
3. Database System Concepts (7th Edition) — A.
Silberschatz, H.F. Korth, S. Sudarshan — McGraw-
Hill Education, 2019

Web Development & Backend:

1. Express.js Official Documentation — OpenJS
Foundation — <https://expressjs.com/>
2. Node.js Documentation — OpenJS Foundation —
<https://nodejs.org/en/docs>
3. MDN Web Docs: HTML, CSS, JavaScript — Mozilla
Foundation — <https://developer.mozilla.org/>
- Authentication & Security:
4. Firebase Authentication Documentation — Google
— <https://firebase.google.com/docs/auth>
5. JSON Web Tokens (RFC 7519) — Internet
Engineering Task Force —
<https://datatracker.ietf.org/doc/html/rfc7519>

Machine Learning & Fraud Detection:

1. scikit-learn: Machine Learning in Python — F.
Pedregosa et al. — Journal of Machine Learning Research,
Vol. 12, pp. 2825-2830, 2011
2. Greedy Function Approximation: A Gradient
Boosting Machine — J.H. Friedman — Annals of
Statistics, Vol. 29, No. 5, 2001

3. Credit Card Fraud Detection Using Machine Learning Techniques — S. Patel, R. Garg — IEEE International Conference on Computing, Communication and Security, 2023
4. Flask Web Development Documentation — Pallets Projects — <https://flask.palletsprojects.com/>

Fintech & Digital Payments:

1. Digital Payment Fraud: Trends and Prevention — Reserve Bank of India — RBI Annual Report 2024
2. UPI Fraud Prevention Framework — National Payments Corporation of India — <https://www.npci.org.in/>

APPENDIX

Stored Procedure — ACID Fund Transfer

```
CREATEPROCEDURE sp_send_money(  
    INp_sender_wallet VARCHAR(10),  
    INp_receiver_wallet VARCHAR(10),  
    INp_amountDECIMAL(15,2),  
    INp_fraud_score DECIMAL(5,2)  
)  
BEGIN  
    DECLAREEXIT HANDLER FOR SQLEXCEPTION  
        BEGINROLLBACK; END;  
    STARTTRANSACTION;  
    --Pessimisticlocking: prevents double-spending  
    SELECTBalance FROM WALLET WHERE Wallet_ID = p_sender_wallet FOR  
  
    UPDATE;  
        SELECTBalance FROM WALLET WHERE Wallet_ID = p_receiver_wallet  
    FORUPDATE;  
  
    --Iffraudscore too high, block without moving money  
    IFp_fraud_score >= 90 THEN  
        INSERTINTO TRANSACTION (...) VALUES (...,'Blocked');  
        COMMIT;  
    ELSE  
        UPDATEWALLET SET Balance = Balance - p_amount WHERE Wallet_ID  
=p_sender_wallet;  
        UPDATEWALLET SET Balance = Balance + p_amount WHERE Wallet_ID  
=p_receiver_wallet;  
        INSERTINTO TRANSACTION (...) VALUES (...,'Success');  
        COMMIT;  
    ENDIF;  
END
```

Key Triggers

```
-- Auto-create fraud alert when score >= 60%
CREATE TRIGGER trg_fraud_on_insert AFTER INSERT ON TRANSACTION
FOREACH ROW
BEGIN

    IF NEW.fraud_score >= 60 AND NEW.is_flagged = TRUE THEN
        INSERT INTO FRAUD_ALERT (Alert_ID, Alert_Type, Risk_Score, Txn_ID)
        VALUES (CONCAT('A', FLOOR(RAND()*99999)), 'HighAmount',
        NEW.fraud_score, NEW.Txn_ID);
    ENDIF;
END

--Auto-log balance changes for audit compliance
CREATE TRIGGER trg_wallet_audit AFTER UPDATE ON WALLET
FOREACH ROW
BEGIN

    IF OLD.Balance != NEW.Balance THEN
        INSERT INTO AUDIT_LOG (Table_Name, Record_ID, Old_Value,
        New_Value)
        VALUES ('WALLET', NEW.Wallet_ID,
        JSON_OBJECT('Balance', OLD.Balance),
        JSON_OBJECT('Balance', NEW.Balance));
    ENDIF;
END
```

ML Feature Computation from MySQL

```
// Backend computes 15 features from live MySQL data before calling
ML model

// Burst detection: how many transactions in last 1 hour?
const [[h1]] = await db.query(
'SELECT COUNT(*) AS c FROM TRANSACTION WHERE
Sender_Wallet_ID=? AND Txn_Time >= NOW() - INTERVAL 1
HOUR',
[senderWalletId]);

// User's historical average (baseline for anomaly detection)
const [[stats]] = await db.query(
'SELECT AVG(Amount) AS avg_amt, STDDEV(Amount) AS
std_amt FROM TRANSACTION WHERE Sender_Wallet_ID=?',
```

```

[senderWalletId]);

// Has sender ever sent to this receiver before?
const [[recv]] = await db.query(
  'SELECT COUNT(*) AS c FROM TRANSACTION WHERE
  Sender_Wallet_ID=? AND Receiver_Wallet_ID=?',
  [senderWalletId, receiverWalletId]);

// Account age in days
const [[age]] = await db.query(
  'SELECT DATEDIFF(NOW(), Created_Date) AS days FROM
  WALLET WHERE Wallet_ID=?',
  [senderWalletId]);

// Time since last transaction
const [[last]] = await db.query(
  'SELECT TIMESTAMPDIFF(MINUTE, MAX(Txn_Time),
  NOW()) AS mins FROM TRANSACTION WHERE
  Sender_Wallet_ID=?',
  [senderWalletId]);

```

Firebase Authentication Flow

```

// Frontend: Google Sign-In → auto-register if new user
async function doGoogleLogin() {
  const result = await firebase.auth().signInWithPopup(new
  firebase.auth.GoogleAuthProvider());
  const email = result.user.email;

  // Check if user exists in MySQL
  const check = await fetch('/api/check-email', { body:
  JSON.stringify({ email }) });

  // New user → auto-register in MySQL (creates USER +
  WALLET)
  if (!check.exists) {
    await fetch('/api/register', { body: JSON.stringify({ email, name:
    result.user.displayName }) });
  }

  authToken = await result.user.getIdToken(); // JWT token for

```

```
backend
}
```

```
// Backend: verify JWT token → check MySQL → grant access
const decoded = await admin.auth().verifyIdToken(token);
const [rows] = await db.query('SELECT * FROM vw_user_wallet
WHERE Email = ?', [decoded.email]);
if (rows.length === 0) return res.status(403).json({ error: 'Not
registered' });
```

SampleAPIResponse—FraudDetection

```
{
  "txn_id": "TXNMNQW5ZMIV6MJ",
  "status": "Blocked",
  "amount": 90000,
  "fraud": {
    "score": 100,
    "risk_level": "CRITICAL",
    "label": "FLAGGED"
  },
  "features_used": {
    "txns_last_1h": 3,
    "avg_amount": 1200,
    "amount_deviation": "12.5 std devs",
    "new_receiver": "Yes (first time)",
    "time": "14:00 (Daytime)",
    "account_age": "365 days"
  }
}
```